Physical AI Workcell: Complete Project Knowledge Base

> **What this file is.** A single, self-contained description of the whole project, written so that a person new to robotics *and* an AI assistant can understand everything about it and answer questions accurately. Every name, number and file path here matches the project as it stands.
>
> **Project owner:** Immanuvel M (Mechatronics Engineering student). **Location on disk:** `~/physical_ai_ws`. **Last updated:** October 2026.

---

0. Instructions for an AI assistant reading this document

You are helping the project owner understand, explain, run, debug, present and extend this project.

1. **Treat this document as the source of truth** for facts about the project: names, numbers, results, file paths, design choices. Do not invent features, results or files that are not here. If a question needs a detail that is not in this document, say so and point to the file where it would be found (see §19, File map) instead of guessing.
2. **The owner is learning.** Explain in plain language first, then add technical depth if asked. Use short sentences, examples and analogies. Define any robotics term you use (see §18, Glossary).
3. **Keep these facts straight; they are easy to get wrong:**
   - The robot is **simulated** (Gazebo). There is no physical robot yet. A real 3D-printed arm (SO-101) is planned, not built (§15).
   - All benchmark results and system tests were produced with the **offline rule-based planner**. The **Claude (LLM) planner is implemented and unit-tested with a mocked client, but has never been run end to end**, because no API key was configured.
   - The benchmark is **76 trials in total: 38 for the AI pipeline, 38 for the traditional baseline**. 84 % = 32/38 and 16 % = 6/38.
   - Perception has **no trained neural network**; it is geometric (depth-based) segmentation.
   - Grasping in simulation uses a **"weld" trick** (a Gazebo detachable joint), explained in §8.6.
   - The software was developed with an **AI coding assistant (Claude Code)** under the owner's direction. If the owner asks how to talk about authorship, encourage honesty: "developed with AI-assisted coding" is accurate, and the owner should be able to explain each part.
4. **For "how do I…" questions**, give the exact commands from §10. Each terminal command must start with sourcing ROS: `cd ~/physical_ai_ws && source /opt/ros/humble/setup.bash && source install/setup.bash && …`
5. **For "why does it…" questions**, check §12 (design decisions) and §13 (problems solved) first.
6. **For interview or presentation prep**, use §17 and keep claims to what §11 measured.

---

1. The project in one minute

Name: Physical AI Workcell – Language-Instructed Pick-and-Place Robot.

What it does: You type an instruction in plain English, for example "*Put the red cube in the blue bin*". A simulated robot arm then:

1. **Sees** the table with an overhead 3D (RGB-D) camera and measures every object's position, size, orientation, colour and shape.
2. **Understands** the instruction and **plans** a short sequence of actions (pick X, place in Y).
3. **Acts**: moves the arm without collisions, grasps the part, places it.
4. **Checks** with the camera that the part really ended up where it should.
5. **Re-plans** automatically if something went wrong (a part was moved, a grasp missed).
6. The whole time, a **safety supervisor** slows the arm when a person is near, stops it when a person is too close, and provides an emergency stop.

Why it matters: Traditional factory robots are programmed with fixed, "taught" positions. If a part is a few centimetres off, they grab empty air. This system looks every time, so it copes with parts in random places, changing light, clutter and unseen objects without reprogramming.

Elevator pitch (30 s): "Traditional robots are taught fixed positions, so they fail when anything moves. Ours looks with a 3D camera every time, understands plain-language instructions, plans the moves, executes them without collisions and checks with the camera that it worked. If something changes mid-task it re-plans. A safety layer slows and stops it near people. In 76 randomised trials it succeeded 84 % of the time versus 16 % for a traditionally programmed robot on the same arm."

---

2. Status at a glance

Item	Status
12 ROS 2 packages, ~4,600 lines of Python + robot/config files	Done
Simulation (Gazebo Fortress) with Franka Panda arm, camera, table, bins, tray, parts, cup, simulated worker	Done
Perception, world model, skills, planner, executor, safety, web console	Done
Test: robot / camera / motion (`phase1_check`)	**15/15** pass
Test: pick and place (`skills_check`)	**12/12** pass
Test: perception (`perception_check`)	**48/48** pass, 1.5 mm mean position error
Test: whole system incl. safety (`system_check`)	**10/10** pass
Unit tests: planner (`pytest`)	**8/8** pass
Benchmark: AI pipeline (offline planner) vs traditional taught cell	**84 % vs 16 %** (38 trials each)
Claude (LLM) planner	Implemented, unit-tested with a mock, **never run end to end** (no API key)
Real hardware (SO-101 3D-printed arm)	Planned: parts list, print files and workcell kit ready; software port not started

Machine it was built on: Ubuntu 22.04, ROS 2 Humble, Gazebo Fortress (ign-gazebo 6.18), MoveIt 2, Python 3.10, 8 CPU cores, 7 GB RAM, AMD integrated GPU (no NVIDIA/CUDA). Anthropic Python SDK 1.13.0 is installed.

---

3. The challenge this project answers

It was built for a "Physical AI" challenge with these requirements (paraphrased): build one connected pipeline from instruction to completed physical task; identify relevant objects and their spatial relations; plan the right sequence of moves; execute safely with actuators; cope with real-world variation (positions, orientations, lighting, clutter, obstacles) without reprogramming; handle tasks it was not set up for; correct itself while running; operate safely near people; clearly outperform a traditional rule-based setup. Deliverables: a prototype or simulation, AI models and software, the perception-to-action workflow, a demo of at least one task, metrics (accuracy, success rate, response time) and documentation (architecture, data/training approach, safety, limitations, scalability).

Requirement	How the project meets it	Evidence
One connected pipeline	Shared world model + closed loop: perceive → plan → act → verify → re-plan	`executor_node.py`; system test A/B
Identify objects / work area	Class-agnostic RGB-D segmentation; bins and trays recognised	perception test 48/48
Spatial relations	Scene graph relations `in`, `on`, `near`; held object	`pai_core/scene.py`
Sequence of moves	Planner outputs a validated plan over 3 skills	`planner.py`, unit tests
Safe execution	MoveIt collision-aware motion; safety supervisor	skills test; system test F/G
Real-world variation	No taught positions; benchmark randomises pose, light, clutter	benchmark
Untrained tasks	Unseen objects detected with no retraining; paraphrases need the LLM planner	benchmark "novel"
Self-correction	Verify every place; re-plan on failure; retry alternative grasps/spots	system test C
Safe near people	Speed & separation monitoring, e-stop, fail-safes	system test F/G
Beats rule-based	84 % vs 16 %	benchmark

4. Robotics basics needed to follow this project

- **ROS 2 (Robot Operating System 2, version "Humble"):** a framework where a robot's software is split into many small programs called **nodes** that run at the same time and talk to each other.
- **Topic:** a named channel a node broadcasts on continuously (like a radio station). Example: the camera publishes images on `/camera/image`.
- **Service:** a quick request → reply between nodes. Example: `/world_model/refresh` ("look now").
- **Action:** a long request that reports progress and can be cancelled. Example: `/pai/skills/pick`.
- **Message (msg) / action definition:** the data format sent over a topic or action.
- **Launch file:** a script that starts many nodes together with their settings.
- **Package:** a folder of related nodes/config that ROS can build (`colcon build`).
- **Gazebo (Fortress):** a physics simulator: virtual robot, camera, objects, gravity, collisions.
- **URDF / xacro:** a file describing a robot's links (rigid parts), joints, limits, masses and meshes.
- **MoveIt 2:** motion-planning library: computes arm motions that avoid collisions; inverse kinematics (IK) finds joint angles that put the gripper at a target pose.
- **ros2_control:** the layer that sends joint commands to the (simulated or real) motors. Uses "controllers", e.g. a JointTrajectoryController that follows a timed list of joint positions.
- **TF:** keeps track of where every frame (robot link, camera) is in space.

- **TCP (tool centre point):** the point between the fingertips; "move to X" means move the TCP to X.
- **RGB-D camera:** gives a colour image plus a depth image (distance per pixel) → 3D points.
- **Planning scene:** MoveIt's model of obstacles. If an object is not in it, MoveIt will happily plan through it.

---

5. Architecture

5.1 Big picture

```
you ──► Operator console (web page, localhost:8080) or CLI
            │  RunInstruction action
            ▼
      EXECUTOR (pai_planner) ── runs the loop: perceive → plan → act → verify → re-plan
        │        │                        │
        │        │ plan                   │ Pick / Place / MoveTo actions
        │        ▼                        ▼
        │   PLANNER                SKILL SERVER (pai_skills) ── MoveIt 2 ── ros2_control ── arm
        │   (Claude or rule)          ▲ safety status (speed scale / stop)
        │                             │
        │ /world_model/refresh    SAFETY SUPERVISOR (pai_safety) ◄── person positions (human_sim)
        ▼                             └── can also hold the arm directly via the controller
   WORLD MODEL (pai_world_model) ── keeps /scene_graph + MoveIt planning scene in sync
        ▲  /perception/refresh
   PERCEPTION (pai_perception) ◄── overhead RGB-D camera (Gazebo)
```

Everything shares **one world model**. The executor never acts on stale information: it perceives before planning, after every place, and before every re-plan.

5.2 The 12 packages (all in `~/physical_ai_ws/src/`)

Package	Job	Key files
`pai_bringup`	One launch file for everything; builds the Gazebo world from a scene YAML; controllers; Phase 1 test	`launch/bringup.launch.py`, `config/scene_default.yaml`, `scripts/make_world.py`, `config/ros2_controllers.yaml`, `config/gazebo_gui.config`, `scripts/phase1_check.py`
`pai_description`	Panda robot model for Gazebo (official meshes + added masses/inertias, ros2_control, `panda_tcp`, sim grasp joints)	`urdf/panda_gz.urdf.xacro`
`pai_moveit_config`	MoveIt settings: SRDF (groups, poses, collision pairs), KDL IK, joint limits, OMPL and Pilz planners, controllers	`config/panda.srdf`, `config/*.yaml`
`pai_interfaces`	Custom messages and actions (§6)	`msg/*.msg`, `action/*.action`
`pai_core`	Shared Python library: geometry, scene-graph helpers, MoveIt planning-scene conversion, Claude client, simulation control	`pai_core/*.py`
`pai_perception`	Camera → list of objects	`pai_perception/segment.py` (algorithm), `perception_node.py` (ROS node), `perception_check.py` (test)

Package	Job	Key files
`pai_world_model`	Tracks objects across looks, relations, held object, MoveIt planning scene	`world_model_node.py`
`pai_skills`	The robot's three actions: pick, place, move_to	`skill_server.py`, `skills_check.py` (test)
`pai_planner`	Planners (Claude + rule), plan validator, executor (the main loop), CLI	`planner.py`, `executor_node.py`, `cli.py`, `test/test_planner.py`
`pai_safety`	Safety supervisor; simulated co-worker / person sensor	`supervisor.py`, `human_sim.py`
`pai_ui`	Browser operator console	`console.py`, `index.html`
`pai_benchmark`	System test, traditional baseline cell, randomised benchmark, report	`system_check.py`, `baseline.py`, `benchmark.py`, `report.py`, `common.py`

5.3 Coordinate conventions

- `world` frame = robot base. **Table top is z = 0**, floor is z = −0.75 m. +x points away from the robot, +y to the robot's left. Units: metres and radians.
- The scene YAML (`scene_default.yaml`) defines the layout and is also the benchmark's reference.

---

6. Interfaces reference

6.1 Messages (`pai_interfaces/msg`)

- **DetectedObject:** `id` (stable track id, e.g. `red_cube`), `label` ("red cube"), `color`, `shape` (box | cylinder | bin | tray | unknown), `category` (part | receptacle | obstacle), `pose` (centre, yaw only), `size` (x, y, z extents in m), `confidence`, `graspable`.
- **Relation:** `subject`, `predicate` (in | on | near), `object`.
- **SceneGraph:** `header`, `objects[]`, `relations[]`, `held_object`.
- **SafetyStatus:** `state` (NORMAL | SLOW | STOP | ESTOP), `speed_scale` (0–1), `min_human_distance` (m), `reason`.
- **TaskStatus:** `instruction`, `state` (IDLE | PERCEIVING | PLANNING | EXECUTING | VERIFYING | REPLANNING | PAUSED | SUCCEEDED | FAILED | NEEDS_CLARIFICATION), `planner`, `plan_json`, `current_step`, `message`.

6.2 Actions (`pai_interfaces/action`)

- **RunInstruction** (`/pai/run_instruction`, and `/pai/baseline/run_instruction` for the baseline): goal `instruction`, `dry_run`; result `success`, `summary`, `plan_json`, `planning_time`, `time_to_first_motion`, `total_time`, `replans`; feedback `state`, `current_step`, `message`.
- **Pick** (`/pai/skills/pick`): `object_id`, or an explicit `pose` + `size` (used by the baseline).
- **Place** (`/pai/skills/place`): `target_id` (a receptacle), or `target_point` + `surface_z` + `rim_z` (explicit taught place, used by the baseline).
- **MoveTo** (`/pai/skills/move_to`): `named_target` = `ready` or `camera_clear`.

6.3 Main topics and services

Name	Type	Purpose
`/camera/image`, `/camera/depth_image`, `/camera/camera_info`	sensor_msgs	Overhead RGB-D camera (640×480, 10 Hz), frame `camera_optical_frame`
`/perception/refresh`	std_srvs/Trigger	Segment the next camera frame
`/perception/detections`, `/perception/debug_image`	SceneGraph, Image	Raw detections, annotated picture
`/world_model/refresh`	Trigger	Fresh perception → updated scene graph (returned as JSON)
`/scene_graph`	SceneGraph (latched)	The shared world state
`/pai/manipulation_state`	String JSON (latched)	Which part is targeted, held, or was just placed
`/safety/status`	SafetyStatus, 20 Hz	Speed scale / stop for the skills
`/safety/humans`	PoseArray, 10 Hz	Person positions (from `human_sim`)
`/safety/estop`	std_msgs/Bool	true = emergency stop (latched until reset)
`/safety/reset`	Trigger	Clear the e-stop
`/safety/human_sim/walk_in`	Trigger	Make the simulated co-worker walk up
`/pai/task_status`	TaskStatus	Live status for the console
`/pai/skills/release_all`	Trigger	Test-harness reset: drop the held part, release all sim grasp joints
`/world/workcell/set_pose`	ros_gz SetEntityPose	Teleport a Gazebo model (tests/benchmark only)
`/world/workcell/dynamic_pose/info`	tf2 TFMessage	Ground-truth object poses (sim grasp + scoring only)

---

7. One instruction, step by step (with code pointers)

Instruction: "*Put the red cube in the blue bin.*"

1. **Console → executor.** `pai_ui/console.py` sends a `RunInstruction` goal. `executor_node.py` `_execute()` starts. State **PERCEIVING**.
2. **Clear the camera view.** `_look()` moves the arm to the `camera_clear` pose (joints `[0, -1.4, 0, -2.9, 0, 1.6, 0.785]`) and fails the task if the arm cannot get there.
3. **Perceive.** `/world_model/refresh` → `/perception/refresh` → `segment.py segment()` finds the objects; `world_model_node._integrate()` assigns stable ids, computes relations, updates MoveIt's planning scene. Takes ~0.8 s.
4. **Plan.** State **PLANNING**. `_make_plan()` calls the Claude planner if available (`planner.py ClaudePlanner.plan`), otherwise `RulePlanner.plan`. Output: `pick(red_cube)`, `place(blue_bin)`. `validate()` dry-runs it. If the instruction is ambiguous → **NEEDS_CLARIFICATION** with a question and no motion.
5. **Execute pick.** State **EXECUTING**. `skill_server._pick()` (details §8.6).
6. **Execute place.** `skill_server._place()` (details §8.6).

7. **Verify.** State **VERIFYING**. `_look()` + perceive; `_verify_place()` checks that the relation `red_cube in blue_bin` exists. If not → failure.
8. **Re-plan on any failure.** State **REPLANNING**: look again, ask the planner for the *remaining* work with the history of what happened; up to `max_replans = 3`.
9. **Finish.** The executor checks that every part the original plan promised was delivered (no "success" if a part vanished), moves to `camera_clear`, reports **SUCCEEDED** with timings, and appends a record to `~/.ros/pai/runs/runs.jsonl`.

Typical timing for one part: ~30 s total; first motion ~1–3 s after the instruction (rule planner).

---

8. Each part in depth

8.1 Simulation world (`pai_bringup`)

- `make_world.py` turns `config/scene_default.yaml` into a Gazebo world file at launch time (`~/.ros/pai/workcell.sdf`). A new layout = edit the YAML, no code changes.
- Default layout (metres; world frame): table 1.2 × 1.4 m centred at (0.35, 0).
  - **blue_bin** 0.18 × 0.18 × 0.08 at (0.30, 0.45); **green_bin** same size at (0.55, 0.45); **kit_tray** 0.26 × 0.18 × 0.025 at (0.45, −0.45). Bin walls 6 mm.
  - **Parts:** red_cube 4 cm at (0.45, 0.05); blue_bolt cylinder r 1.5 cm × 7 cm at (0.58, −0.12); yellow_block 6 × 3 × 3 cm at (0.36, −0.16); green_nut cylinder r 2 cm × 2 cm at (0.62, 0.10).
  - **Pool of "unseen" parts** parked on the floor out of view until a test brings them in: orange_box, purple_cylinder, cyan_block, red_cylinder.
  - **Obstacle:** coffee_cup cylinder r 4 cm × 10 cm at (0.50, 0.18). Never to be picked.
  - **Human:** "worker" mannequin parked at (2.2, −1.6); kinematic, no collisions.
- **Camera:** static overhead RGB-D at (0.45, 0, 1.20) looking straight down, 640 × 480, horizontal FOV 1.05 rad, 10 Hz.
- **Gazebo GUI:** side panels hidden by default (`gui_panels:=true` shows them; layout file `config/gazebo_gui.config`).

8.2 Robot model (`pai_description`) and control

- Franka Panda (7 joints + 2-finger gripper) using the official meshes from `moveit_resources_panda_description`. The upstream model has no masses, so approximate inertials were added (fine for a position-controlled arm).
- `panda_tcp` frame 0.1034 m below the hand (between the fingertips).
- **Controllers** (`ros2_controllers.yaml`, 250 Hz): `joint_state_broadcaster`, `panda_arm_controller` (JointTrajectoryController, position), `panda_hand_controller` (GripperActionController). Gripper "open" = **0.035 m** per finger, deliberately *not* the 0.04 m limit (see §13).
- **Sim grasp joints:** one Gazebo DetachableJoint per pickable object, parent link `panda_link7` (Gazebo merges the hand into link 7), topics `/pai/grasp/<object>/attach` and `/detach`.

8.3 MoveIt configuration (`pai_moveit_config`)

- Planning group `panda_arm` (chain `panda_link0` → `panda_link8`), group `hand`; named states `ready`, `camera_clear`, `open`, `close`. KDL inverse kinematics.
- Two planning pipelines: **Pilz industrial motion planner (PTP)** for direct, predictable point-to-point moves, and **OMPL (RRTConnect)** as fallback that routes around obstacles.
- MoveIt's own automatic re-planning is **turned off** (it would fight the safety hold, §13).

8.4 Perception (`pai_perception`)

Algorithm in `segment.py` (pure NumPy/OpenCV, no learned weights):

1. Back-project the depth image to 3D points in the world frame using the camera intrinsics and TF.
2. **Robot filter:** mask pixels within capsules around the robot's links (radius 12 cm near base and around wrist/hand, 9 cm elsewhere). Any segment *touching* the robot mask is discarded.
3. Keep points on the table (bounds `[-0.22, 0.92] × [-0.67, 0.67]`) that rise **8 mm to 30 cm** above it; clean with a morphological opening; split into connected components (≥ 25 pixels).
4. **Receptacles:** a segment with a footprint side > 12 cm. If its interior is well below its rim (> 35 % of interior points below half the rim height) it is a **bin**, otherwise a **tray**.
    - Contents of a bin: everything inside the walls (12 mm inset) and above the floor, found by **geometry only** (so a blue bolt in a blue bin is still seen).
    - Contents of a tray: points above the tray's dominant surface height.
5. **Parts:** footprint, centre and yaw from a minimum-area rectangle of the **top face** (avoids perspective spill of tall parts); height from the supporting surface; **shape** = cylinder if the top face fills < 86 % of its bounding box and is roughly square, else box; **colour** named from the top face's median colour in HSV, with white/grey judged relative to the table's brightness (robust to dim light). Category = part if its narrow side ≤ 6.4 cm and height < 12 cm, else obstacle.
6. Optional `perception_backend:=claude`: Claude vision names each segment ("coffee mug"), cached. Geometry keeps the final say on what is graspable or a receptacle.

Measured: 48/48 checks, **1.5 mm mean / 2.2 mm max position error** over 38 detections, ~0.8 s per look.

8.5 World model (`pai_world_model`)

- On `/world_model/refresh`: asks perception, then folds detections into **tracks**:
    1. a just-placed part keeps its id if a same-colour detection is within 10 cm of its release point;
    2. otherwise greedy nearest match on same colour and receptacle/part type within 10 cm;
    3. new objects get ids like `red_cube`, `red_cube_2`;
    4. the held part (hidden by the robot filter) is kept.
- **Safeguard:** new detections closer than 10 cm (+ half their size) to a robot link are rejected as perception artefacts (prevents the "robot in collision with itself" deadlock, §13).
- Computes relations (`in` bin, `on` tray or table, `near` < 8 cm) and publishes `/scene_graph`.
- Syncs MoveIt's planning scene (`pai_core/planning_scene.py`): table (top 2 mm below z = 0), bins as floor + 4 walls (exact), tray exact, parts padded **1 cm**, obstacles padded **2 cm**. The held part, the part being grasped, and a just-placed part (until re-seen) are excluded.

8.6 Skills (`pai_skills/skill_server.py`)

Safety-aware motion (`_run_motion`): every motion waits while the supervisor says STOP/ESTOP, scales speed by the supervisor's `speed_scale` (base `max_velocity` = 0.35), cancels and re-plans if the allowed speed drops mid-motion, and **redoes any motion that overlapped a protective stop** (tracked by a stop counter). Free-space moves try **Pilz PTP first, OMPL if blocked**. Every target is checked against a workspace box: x 0.15–0.85, y ±0.62, z 0.003–0.65 m.

Pick (`_pick`):

1. Refuse if already holding something, if the part is a receptacle, if its narrow side > 6.4 cm ("too wide"), or if it sits too deep inside a bin for the hand to reach.
2. Grasp height: about 2.5 cm of finger overlap, fingertips ≥ 1 cm above the supporting surface.

3. Wrist angle candidates: cylinders every 30° (12), square parts every 90° (4), others 0°/180°; ranked by how far the 22 cm hand body stays from tall neighbours; the best 4 are tried.
4. Move above the part (at least 18 cm high) with the part still in the collision model; the arm must really arrive (within 3 cm, one retry).
5. Remove the part from the collision model; straight-line descent (refused if it would move more than 3 cm sideways); wait until the TCP is within 3 mm of the target.
6. **Simulation:** confirm (from Gazebo ground truth) the part is between the fingers, **weld** it to the hand, then close the fingers to the part's width. **Real hardware path:** close until the fingers stall; fully closed means "missed".
7. Lift 15 cm; check the part came up with the hand.

Place (`_place`):

1. Candidate spots on a 9 × 9 grid inside the receptacle × 2 wrist angles, scored by clearance from parts already there and from the hand body; best first, up to 5 tried.
2. Release height = surface + part height below the TCP + 1.2 cm; in a bin never lower than rim − 3 cm (the hand must stay above the walls; the part drops the last few cm).
3. Move above, straight-line lower, un-weld, open (in a bin only as wide as the part + 1 cm, so the fingers cannot jam on the walls), retreat, and verify the part did not follow the hand.

Move to: named joint poses `ready` and `camera_clear`.

8.7 Planner (`pai_planner/planner.py`)

- **Plan format (JSON):** `status` (ok | clarify | impossible), `reasoning`, `question`, `steps[]` of `{skill: pick|place|move_to, object_id, target_id, named_target, why}`.
- **Validator `validate()`:** symbolically executes the plan: ids must exist; pick only graspable `part`s with an empty gripper; place only into a `receptacle` while holding; plan must not end holding something.
- **Claude planner:** model `claude-opus-5-5`, effort `medium`, JSON-schema structured output, server-side refusal fallback (`fallbacks: "default"`, beta `server-side-fallback-2026-07-01`). Sends `{instruction, scene, history}`; one repair round if the validator rejects the plan. Credentials: `ANTHROPIC_API_KEY` or the file `~/.config/pai/anthropic_api_key`. Any API error → falls back to the rule planner (in `auto` mode). **Not yet run end to end.**
- **Rule planner (offline):** keyword parser. Handles "sort … by colour" (each part → receptacle of the same colour) and "put/move X (and Y) in/on/into/onto Z". Synonyms: cube/block/box/brick = box; bolt/cylinder/peg/pin/nut/rod = cylinder; bin/container/basket/box (destination) = bin; tray/kit/pallet = tray. Ambiguous or unknown references → asks a question. It cannot understand paraphrases such as "crimson cube goes with the other blue things".
- Planner selection is the executor parameter `planner` = `auto` (Claude if a key works, else rule), `claude` or `rule`, switchable at runtime: `ros2 param set /executor planner rule`.

8.8 Executor (`pai_planner/executor_node.py`)

The main loop described in §7. Parameters: `planner` (auto), `model` (claude-opus-5-5), `effort` (medium), `max_replans` (3), `log_dir` (`~/.ros/pai/runs`). Never reports success it did not verify; rejects a new instruction while busy; publishes `/pai/task_status`.

8.9 Safety (`pai_safety`)

- **Supervisor zones** (distance = horizontal distance from a person to the nearest robot link minus a 0.25 m body radius):
  - more than 0.9 m → **NORMAL**, full speed;
  - 0.45–0.9 m → **SLOW**, 30 % speed;

    - under 0.45 m → **STOP** (protective stop); resumes automatically after 1 s clear;
    - e-stop → **ESTOP**, latched until `/safety/reset`;
    - person-sensor data older than 1 s, or robot pose unknown → **STOP** (fail-safe).
- **Direct hold:** on STOP/ESTOP the supervisor sends the arm controller a one-point trajectory at the current joint positions (repeated at 5 Hz), bypassing MoveIt. Measured: an arm moving at 1.2–1.5 rad/s halts in **0.28–0.36 s** and holds with no measurable drift.
- **Skill server fail-safe:** with no supervisor heartbeat for 1 s, nothing moves.
- **Simulated co-worker** (`human_sim.py`): publishes the mannequin's position on `/safety/humans` (standing in for a safety laser scanner); `walk_in` makes it walk to (1.4, 1.2), then reach in at (0.75, 0.62) for 8 s, then leave.

8.10 Operator console (`pai_ui`)

Web page at `http://localhost:8080` (local only), served by `console.py` with Python's standard library. Shows: instruction box + example buttons, live camera, last perception picture, current plan with the active step, scene graph table and relations, safety badge, E-STOP and Reset, "Simulate a co-worker walking up", and a log. API: `GET /api/state`, `/api/camera.jpg`, `/api/perception.jpg`; `POST /api/run`, `/api/cancel`, `/api/estop`, `/api/reset`, `/api/walk_in`.

8.11 Benchmark, baseline and tests (`pai_benchmark`)

- **Traditional baseline (`baseline.py`):** three taught programs with fixed coordinates from the default layout: P1 red cube → blue bin, P2 sort (bolt → blue bin, nut → green bin), P3 kit (yellow block + bolt → tray slots). Selected by keywords; open loop: no camera, no verification, no re-planning. Uses the **same skills and motion planner** as the AI pipeline, so the comparison isolates task understanding and adaptation.
- **Benchmark (`benchmark.py`):** seeded random scenes; parts at least 9 cm apart and 12 cm from the cup. Conditions:
    - **nominal:** the taught layout;
    - **pose:** random positions and yaws;
    - **lighting:** plus random light direction and intensity;
    - **clutter:** plus 2 distractor parts and the cup moved;
    - **disturbance:** a goal part is teleported 1.5 s after the robot starts reaching for it;
    - **novel:** unseen objects and paraphrases.
- Tasks:
    - **single:** "Put the red cube in the blue bin."
    - **sort:** "Sort the parts by colour: blue parts go in the blue bin and green parts in the green bin."
    - **kit:** "Prepare a kit: place the yellow block and the blue bolt on the kit tray."
    - Novel tasks:
    - "Drop the orange box into the green bin."
    - "The purple cylinder and the cyan block belong on the kit tray."
    - "That crimson cube goes with the other blue things."
    - "Load the bolt and the yellow bar onto the kitting tray, please."
- **Scoring:** a trial succeeds only if every goal holds in Gazebo's ground truth afterwards **and** no other part moved more than 3 cm.
- Report (`report.py`) → `docs/RESULTS.md`, `docs/success_by_condition.png`, `docs/timing.png`.

---

9. Configuration reference

Launch arguments (`ros2 launch pai_bringup bringup.launch.py <arg>:=<value>`):

Argument	Default	Meaning
`headless`	false	Gazebo without a window
`rviz`	true	RViz with the MoveIt panel (use `rviz:=false` to save memory)
`gui_panels`	false	Show Gazebo's side panels
`moveit`, `skills`, `perception`, `safety`, `executor`, `ui`, `baseline`	true	Start that part
`planner`	auto	auto / claude / rule
`perception_backend`	geometric	geometric / claude
`scene`	`config/scene_default.yaml`	Scene YAML to build the world from

Important parameters:

- Skill server: `max_velocity` 0.35, `max_acceleration` 0.35, `approach_height` 0.12, `lift_height` 0.15, `require_safety` true, `safety_pause_timeout` 120 s, `workspace` box.
- Supervisor: `stop_distance` 0.45, `slow_distance` 0.90, `slow_scale` 0.3, `body_radius` 0.25, `sensor_timeout` 1.0, `resume_delay` 1.0.

---

10. How to run everything

All commands start with: `cd ~/physical_ai_ws && source /opt/ros/humble/setup.bash && source install/setup.bash &&`

Goal	Command (after the prefix above)
Build (after code changes)	`colcon build --symlink-install`
Start the demo	`ros2 launch pai_bringup bringup.launch.py rviz:=false` then open http://localhost:8080
Instruction from a terminal	`ros2 run pai_planner run_instruction "Put the red cube in the blue bin"` (add `--dry-run` to only plan)
Move a part mid-task (demo of re-planning)	`ros2 service call /world/workcell/set_pose ros_gz_interfaces/srv/SetEntityPose "{entity: {name: red_cube, type: 2}, pose: {position: {x: 0.38, y: -0.03, z: 0.021}}}"`
Test 1: robot/camera/motion	start bringup (`headless:=true rviz:=false` is fine), then `ros2 run pai_bringup phase1_check.py`
Test 2: pick & place	bringup with `perception:=false executor:=false`, then `ros2 run pai_skills skills_check`
Test 3: perception	bringup, then `ros2 run pai_perception perception_check`
Test 4: whole system	bringup, then `ros2 run pai_benchmark system_check` (~15 min)
Planner unit tests	`PYTEST_DISABLE_PLUGIN_AUTOLOAD=1 python3 -m pytest src/pai_planner/test`

Goal	Command (after the prefix above)
Benchmark (starts its own simulator)	`scripts/run_benchmark.sh` (~80 min); with Claude: `scripts/run_benchmark.sh --systems claude --trials 2`
Save an API key (optional)	`mkdir -p ~/.config/pai && read -s -p "Anthropic API key: " k && echo "$k" > ~/.config/pai/anthropic_api_key && chmod 600 ~/.config/pai/anthropic_api_key && unset k`

Notes: only Gazebo has a window; everything else runs in the background and is controlled from the web page. The robot does nothing until it gets an instruction. Stop with Ctrl+C; restart to reset the parts. Never run the benchmark and the demo at the same time (7 GB RAM; two simulators interfere).

---

11. Results

11.1 Tests (all on the same final code)

Test	Result	What it covers
`phase1_check`	15/15	joint states, controllers, camera streams and frame, camera→world projection of the red cube, MoveIt joint and TCP goals (TCP error 1.5–2.8 mm), 3 gripper cycles
`skills_check`	12/12	red cube → blue bin, blue bolt → kit tray, yellow block → kit tray (verified in Gazebo), refusal of the 8 cm cup
`perception_check`	48/48	all objects with correct colour/shape/category, no phantoms, latency < 2 s, stable ids, unseen objects, same-colour parts inside bins, in/on relations, dim side lighting; 1.5 mm mean error
`system_check`	10/10	A: single part, B: two parts with synonyms, C: part moved mid-task → re-plan → success, D: ambiguous → asks, E: cup request → refused, F: person nearby → SLOW + STOP, arm halts, resumes and completes, G: e-stop halts, task completes after reset
planner pytest	8/8	parsing, synonyms, sort, ambiguity, missing object, already-done, validator, Claude repair loop (mocked)

11.2 Benchmark (38 trials per system)

Condition	AI pipeline (offline rule planner)	Traditional taught cell
nominal	100 % (6/6)	100 % (6/6)
pose	100 % (6/6)	0 % (0/6)
lighting	100 % (6/6)	0 % (0/6)
clutter	83 % (5/6)	0 % (0/6)
disturbance	100 % (6/6)	0 % (0/6)
novel	38 % (3/8)	0 % (0/8)
overall	**84 % (32/38)**	**16 % (6/38)**

Other metrics: goals achieved 85 % vs 16 %; reported success with a wrong end state 0 vs 0; trials that needed re-planning 9 (AI); clarification requests 4 (AI); median total time of a successful trial 57 s vs 52 s (the AI checks its work with the camera, the baseline does not).

AI pipeline failures explained:

- 2 trials (clutter kit, novel object): no collision-free straight-down approach to a part next to other objects; stopped safely after 3 re-plans.
- 4 trials (paraphrases): the offline planner could not interpret the sentence and asked for clarification without moving. This is the job of the Claude planner, not yet measured.

Baseline failures: away from the taught layout it grasps where parts used to be ("grasp missed") or collides with parts that are now in the way.

---

12. Design decisions and why

Decision	Why
The LLM chooses among 3 skills and measured object ids, never joint angles or coordinates	Keeps it safe, explainable and checkable; geometry handles millimetres, the LLM handles language
Every plan is validated before motion	Catches LLM mistakes (wrong ids, picking an obstacle)
Geometric perception instead of a trained detector	Works on unseen objects with no data collection; runs on CPU; millimetre accuracy from depth
One shared world model	Makes it one pipeline; every module sees the same state
Verify after every place	Self-correction needs to know when something went wrong; prevents false success
Arm moves out of view before perceiving	The arm hides parts and can be mistaken for objects
Pilz PTP first, OMPL fallback	OMPL's random paths swing widely and clipped the cup; Pilz paths are short and predictable
Safety supervisor holds the controller directly	Cancelling through MoveIt took > 2 s; direct hold stops in ~0.3 s
Fail-safe defaults (no heartbeat / no sensor → no motion)	Standard industrial practice
Sim "weld" grasp	Gazebo finger-contact physics squeezes small light parts out; a real gripper does not need this
Scene YAML generates the world	New layouts without code changes; same file is the benchmark reference
Baseline uses the same skills	Fair comparison: only understanding/adaptation differ, not motion quality
Offline rule planner	Works without internet/API key; serves as an ablation

---

13. Problems solved along the way (good interview material)

Problem	Cause	Fix
Gripper stuck after first open/close	Gazebo (DART) velocity-driven joints parked exactly on a limit can stick	Open = 0.035 m, close targets ≥ 0.001 m (off both limits)
Arm dragged / yanked, contorted joints	Gazebo DetachableJoints start attached; release messages sent before the plugin loaded were lost	Release only after joint states flow, 5 rounds; release before teleporting; skills wait until released
Parts knocked over during approach	Target removed from the collision model before the transit move	Remove it only for the final straight-down descent

Problem	Cause	Fix
Fingers closed on air	Controller reports "done" before the simulated arm has arrived	Wait until the TCP is within 3 mm (grasp) of target
Arm "arrived" 0.78 m away, then swept across the table	Arm stuck (fingers jammed on a bin wall), skill continued, straight-line move went sideways	Gentle release in bins; transit must arrive (3 cm); straight-line moves refused if > 3 cm sideways
Hand hit bin walls	Real hand body starts 3.8 cm above the TCP (measured with MoveIt)	Release ≥ rim − 3 cm; refuse parts too deep in a bin
Blue bolt invisible in blue bin	Bin contents found by colour difference	Find contents by geometry only
Robot "in collision with itself", every plan rejected (deadlock)	Perception saw parts of the arm as grey blocks in an unusual pose	Perceive only from `camera_clear` (checked); drop segments touching the robot; reject detections on robot links
False "success" with a part missing	Part fell out of view; re-planner found "nothing left"	Success requires every originally committed part to be delivered
E-stop ignored from ordinary publishers	QoS mismatch (transient-local subscriber)	Supervisor subscribes with volatile QoS
Supervisor crashed on first state change	rclpy forbids one log call site with changing severities	Separate log calls
Arm kept moving 33 s during STOP	MoveIt auto-replan re-sent motions that fought the safety hold	MoveIt replan off; any motion overlapping a stop is redone by the skill
Coffee cup bumped	Wide OMPL paths + simulated tracking lag	Pilz PTP first; 2 cm obstacle margin; parts ≥ 12 cm from the cup in benchmark scenes
Short part next to the tall cup unreachable	22 cm hand body	Rank 12 wrist angles by hand clearance (documented limitation remains)
API key "missing" crash	SDK 1.x only reports missing credentials at request time	Detect credentials at start-up; any SDK error falls back to the rule planner

---

14. Limitations (state these honestly)

- **Simulation only.** No physical robot has been built yet.
- **Simulated grasping** uses a weld once the part is confirmed between the fingers.
- **Simulated person sensing** (a real cell needs a safety laser scanner or similar).
- **Top-down grasps only.** No tilted grasps or regrasping; parts deep in an 8 cm bin, or within ~12 cm of a tall obstacle, are often unreachable.
- **Perception assumptions:** one calibrated overhead camera, flat table, parts ≥ 8 mm tall, box- or disc-like footprints, no stacking, coarse colour names.
- **Approximate robot masses** (the arm is position-controlled, so tasks are unaffected).
- **Claude planner not measured end to end.** Needs internet, adds seconds per plan, costs roughly US$0.03–0.05 per plan at list prices.
- **Not safety-certified.** A real factory needs certified safety hardware (safety controller, scanner, risk assessment to ISO 10218 / ISO/TS 15066).

---

15. Hardware plan (SO-101, not built yet)

- **Why not the Panda:** an industrial arm (~US$30k) that cannot be 3D printed.
- **Chosen arm: SO-101** follower arm by TheRobotStudio (open source, Apache-2.0), 5 joints + gripper, ~30 cm reach, light payload. Files (trimmed copy) in `~/physical_ai_ws/hardware/SO-ARM100/`: print plates `STL/SO101/Follower/` (Ender, Prusa, Bambu A1 mini), individual parts `STL/SO101/Individual/`, CAD `STEP/SO101/` (`SO101 Assembly.step` = whole arm with motors), simulation model `Simulation/SO101/so101_new_calib.urdf` (includes STS3215 servo meshes).
- **Motors:** 6 × **Feetech STS3215, 7.4 V, 1/345 gear (C001)**, ~US$13.89 each. Optional stronger 12 V / 30 kg·cm version needs a 12 V ≥ 5 A supply. Buy all six in the same version.
- **Shopping list (one arm, official BOM ≈ US$122):** 6 servos, Waveshare serial bus servo board, 5 V power supply (for the 7.4 V servos), USB-C cable, table clamps, screwdrivers. Also needed for this project: an RGB-D camera (e.g. Intel RealSense D435/D405 or Orbbec), a camera arm/tripod, a latching e-stop switch in the servo power line, coloured PLA+.
- **Printing:** PLA+, 0.4 mm nozzle, 0.2 mm layers, 15–20 % infill, supports, ~16 h for the arm. **Workcell kit** designed for this project (`hardware/workcell_print/`, regenerate with `make_stl.py`): blue and green bins 80 × 80 × 35 mm, grey kit tray 110 × 80 × 8 mm, red cube 25 mm, yellow block 35 × 18 × 18, blue bolt Ø16 × 35, green nut Ø24 × 12 with Ø10 hole, orange box 30 × 20 × 20 (print each in its colour; perception names parts by colour).
- **Comfortable working zone:** ~12–24 cm from the arm's base (upper arm 11.6 cm, forearm 13.5 cm).
- **Assembly:** official guide https://huggingface.co/docs/lerobot/so101 (set each servo's ID 1–6 before assembly).
- **Software port (to do):** H1 SO-101 in the simulator + MoveIt for 5 joints (no hardware needed); H2 ros2_control driver for the Feetech servos; H3 real camera driver + calibration; H4 tests on the real arm. Perception, world model, planner, executor, safety and console carry over.
- Opening the STEP in CAD: STEP files contain no mates; parts are positioned but not jointed. In SolidWorks, if the view is blank, disable "3D Interconnect" (Tools → Options → Import) and re-open.

Full guide: `hardware/HARDWARE.md`.

---

16. Troubleshooting

Symptom	Likely cause / fix
Only Gazebo opens	Normal: other parts run in the background. Open http://localhost:8080 and press Run
`ros2: command not found` / package not found	Run the source prefix in §10 in that terminal
Console page does not load	Bringup not running or still starting (wait 30–40 s); check the terminal for errors
Arm does not move after Run	Check the safety badge: STOP/ESTOP → press Reset; "person sensor silent" → `human_sim` not running
Says it needs clarification	The rule planner could not parse the sentence; use the example phrasing or add the Claude key
"too deep inside … bin" / "too wide to grasp"	Correct refusals (§14)
Parts not reset between runs	Stop (Ctrl+C) and relaunch

Symptom	Likely cause / fix
Very slow / choppy	Close other apps; use `rviz:=false`; never run two simulators
Planner says Claude unavailable	No key or `anthropic` not installed; it falls back to the rule planner
Tests fail right after a code edit	Rebuild with `colcon build --symlink-install` and restart bringup

---

17. Likely questions (judges, interviews) with answers

- **Did you train a neural network?** No task-specific training. Perception is geometric segmentation of depth data; the planner is a general LLM used zero-shot (plus an offline rule planner). Test scenes are generated by seeded randomisation; every run is logged and could become training data later.
- **What makes it one pipeline?** One shared world model and one closed loop with verification after every step; failures feed back into re-planning.
- **How do you know results are real?** Scored on Gazebo's ground truth, not the robot's belief; a trial counts only if all goals hold and nothing else was knocked > 3 cm; zero false successes.
- **What if the LLM makes a bad plan?** It can only pick from 3 skills and measured ids; the validator dry-runs every plan; one repair round, otherwise refused.
- **Which planner produced the 84 %?** The offline rule planner. The Claude planner is implemented but not yet benchmarked.
- **How fast does it stop for a person?** Protective stop within 0.45 m; measured halt 0.28–0.36 s from 1.2–1.5 rad/s, via a direct hold command to the controller.
- **Why simulation?** Safe, repeatable, and allows a randomised benchmark with ground truth; the hardware port to the SO-101 is planned.
- **Hardest bug?** E.g. the deadlock where perception saw the robot itself as objects (§13), or the safety hold being fought by MoveIt's automatic re-planning.
- **Could it run on a real robot?** Yes, with a real arm driver and camera; skills, planner, world model and safety stay; safety would need certified hardware for production.

---

18. Glossary

Action server: a node that executes long tasks with progress feedback. **Baseline**: the traditional taught robot used for comparison. **Benchmark**: many randomised trials scored automatically. **Bin / tray**: walled vs flat receptacle. **Collision model / planning scene**: MoveIt's map of obstacles. **DetachableJoint**: Gazebo feature used to weld a part to the hand. **E-stop**: emergency stop. **Ground truth**: the simulator's exact object positions. **IK**: inverse kinematics. **JTC**: JointTrajectoryController. **Latched / transient-local**: a topic that delivers its last message to late subscribers. **LLM**: large language model (Claude). **OMPL**: sampling-based motion planners. **Pilz PTP**: deterministic point-to-point planner. **QoS**: quality-of-service settings of a topic. **RGB-D**: colour + depth camera. **Scene graph**: list of objects + relations. **Skill**: one robot capability (pick, place, move_to). **SSM**: speed and separation monitoring. **TCP**: tool centre point. **Validator**: code that dry-runs a plan. **World model**: the shared, up-to-date scene graph.

---

19. File map

```
~/physical_ai_ws/
├── README.md                     setup, commands, results summary
├── docs/
│   ├── PROJECT_KNOWLEDGE_BASE.md this file
│   ├── TECHNICAL_REPORT.md       formal write-up for the challenge
│   ├── RESULTS.md                benchmark results (generated), benchmark.csv raw data
│   ├── DEMO_SCRIPT.md            5-minute demo walkthrough
│   ├── project_guide.html        beginner-friendly visual guide
│   └── *.png / *.jpg             charts, console and camera screenshots
├── scripts/run_benchmark.sh      runs the benchmark headless and writes the report
├── hardware/
│   ├── HARDWARE.md               SO-101 shopping list, printing, wiring, port plan
│   ├── SO-ARM100/                trimmed official SO-101 design files (see TRIMMED.md)
│   └── workcell_print/           printable bins, tray, test parts + make_stl.py
├── src/                          the 12 ROS 2 packages (§5.2)
├── build/ install/ log/          colcon build outputs (install/ is needed to run)
```

Runtime files: generated world `~/.ros/pai/workcell.sdf`; run log `~/.ros/pai/runs/runs.jsonl`; optional API key `~/.config/pai/anthropic_api_key`.

Other deliverables outside the project: LinkedIn video `~/Videos/linkedin_demo.mp4` (45 s, 1080p, captions, results card); resume `~/Downloads/IMMANUVEL_CV_ROS2_v2.pdf`.